

Rapport de Tests - Projet Minishell

EL AZZOUZI AYOUB, 1SN, Groupe H.

Ce document résume de manière synthétique les principaux tests effectués lors du développement du projet Minishell en TP, ainsi que les comportements observés à chaque étape clé.

TP1 : Gestion de processus et exécution de base

Concernant l'étape 4 (tâche de fond avec '&'), j'ai eu le problème classique mentionné dans le sujet où le prompt réapparaissait trop tard si on lançait un sleep long en tâche de fond suivi d'un sleep court en premier plan. J'ai corrigé ça en utilisant un waitpid spécifique au lieu d'un wait global.

Test : Lancement de ls -l puis d'un sleep 3.

Résultat : Le minishell affiche bien les fichiers, puis pour le sleep, le prompt ne réapparaît qu'après les 3 secondes. L'exécution séquentielle fonctionne.

TP2 : Signaux et contrôle de la terminaison

Ce TP s'est concentré sur la récupération asynchrone des processus fils via SIGCHLD pour éviter les processus zombies, et sur la gestion des états (suspension/reprise).

Test (waitpid & WNOHANG) : Lancement d'un sleep 2 & en arrière-plan puis attente sur le prompt.

Résultat : Le traitement de signal intercepte bien la fin du processus. Le message "[Processus terminé]" s'affiche spontanément sans bloquer le shell, grâce à l'option WNOHANG.

Test (Problème avec pause) : Remplacement de l'attente active par pause()

Résultat : Comme prévu, on tombe sur le problème de concurrence évoqué en cours. Si la commande (ex: ls) est très rapide, le signal SIGCHLD arrive avant que le père n'exécute pause(), ce qui bloque indéfiniment le minishell.

Test (Suspension/Reprise) : Lancement de sleep 100 & , puis utilisation de kill - STOP et kill -CONT dans un autre terminal.

Résultat : Le handler détecte parfaitement les changements d'états grâce aux macros WIFSTOPPED et WIFCONTINUED, affichant les messages de suspension et de reprise dans le terminal du minishell.

TP3 : Signaux et masquage

Pour les tâches de fond, le détachement avec `setpgp()` fait bien le travail. Un Ctrl-C ne tue plus un processus lancé avec '&'.

Test : Frappe de Ctrl-C pendant qu'un `sleep 10` tourne.

Résultat : Avant modification, le minishell se fermait complètement. Après l'ajout de `sigprocmask`, le minishell survit et affiche bien le prompt à nouveau, tandis que la commande `sleep` s'arrête.

TP4 : Fichiers et commandes internes

Pour `cd` et `rep`, les tests sont concluants. Le test de `cd` sans arguments ramène bien au répertoire racine, et `rep` affiche le contenu du répertoire courant, un peu comme un `ls` basique.

Test : `ls -l > out.txt`

Résultat : Rien ne s'affiche dans la console, et un `cat` sur `out.txt` montre bien le résultat. Ça marche aussi avec l'entrée standard.

TP5 : Tubes et pipelines

La partie avec la gestion des descripteurs pour les tubes

Test : `ls | grep ".c" | wc -l`

Résultat : J'ai un peu galéré avec la fermeture des tubes inutilisés dans chaque processus (sinon `wc` restait bloqué indéfiniment à attendre des données). Après correction, le résultat final s'affiche correctement et le programme redonne la main

TP6 : Contrôle des E/S et Couplage Mémoire

La dernière partie visait à manipuler les appels systèmes non-bloquants et le partage de mémoire (`mmap`) entre processus.

Test (`select` vs `fcntl`) : Exécution de `filtrage_fcntl` puis `filtrage_select`.

Résultat : Avec `fcntl`, le programme fonctionne mais on remarque une attente active (le CPU tourne beaucoup car la boucle n'arrête pas de vérifier les descripteurs). Avec `select`,

le programme "dort" vraiment jusqu'à ce qu'une touche soit pressée ou que des données arrivent dans le tube. C'est beaucoup plus propre.

Test (Couplage mémoire - `mmap`) : Exécution du programme testant `MAP_SHARED` et `MAP_PRIVATE` entre un père et son fils.

Résultat : Les modifications apportées par le père en mode `SHARED` sont bien visibles dans le fichier (et par le fils). En revanche, quand le fils modifie la page 2 avec le caractère 'd' (mode `PRIVATE`), c'est bien une copie locale : le fichier d'origine n'est pas altéré, validant le mécanisme de Copy-On-Write (COW).